Author: Brendan Bianchi

SIMD Advantage Profiling — Project Report

AI Usage Statement

Claude was used for code creation, plot generation, assistance in data analysis, and writing support.

Abstract

This project measures how much faster simple numeric kernels become when g++ auto-vectorizes them, and identifies the conditions under which that advantage shrinks or disappears. Four kernels (SAXPY, dot product, element-wise multiply and a 3-point stencil) were compiled as scalar, SSE2, AVX2 and Zen 2–tuned builds and timed on a single pinned core of an AMD Ryzen 5 3600 while the working set was swept from L1-resident (4 KiB) to DRAM-resident (1 GiB). In L1, AVX2 SAXPY runs 6.8× faster than scalar, but the gain falls to 4.8× in L2, 3.8× in the last-level cache and 1.1× in DRAM, where memory bandwidth rather than arithmetic sets the speed. A dot product whose sum may be reordered reaches exactly the 8× lane count, while the plain dot product, which g++ reports as vectorized, runs no faster than scalar. Vectorized strided access can be slower than scalar, down to 0.21× at stride 8. The SIMD advantage therefore tracks lane count only while data stays in cache and access is contiguous, and compiler vectorization reports must be confirmed by disassembly and measurement.

1. Introduction and background

1.1 Motivation

Single Instruction, Multiple Data (SIMD) execution applies one instruction to several data elements, called lanes, held in a wide register. On x86, SSE2 provides 128-bit registers (four float32 lanes), AVX2 provides 256-bit registers (eight float32 lanes) together with fused multiply-add (FMA), and AVX-512 widens registers to 512 bits. The AMD Ryzen 5 3600 used here (Zen 2) supports SSE2 through AVX2 and FMA but not AVX-512, so AVX2 is the widest instruction set studied.

In principle, an eight-lane instruction does eight times the work of a scalar one. In practice that ideal is rarely reached. The processor must also move the data, and the bandwidth available falls at each level of the memory hierarchy. Vector loads and stores that are not aligned to cache lines cost extra. Loop lengths that are not multiples of the vector width leave remainder elements. Non-contiguous access patterns force the compiler to assemble vectors element by element. This project measures how much each of these factors erodes the theoretical advantage.

1.2 Kernels studied

Kernel	Operation	FLOPs / element	Compulsory bytes / element (f32)	Arithmetic intensity (FLOP/B)
SAXPY	y[i] = a·x[i] + y[i]	2	12	0.167
Dot (plain)	s += x[i]·y[i]	2	8	0.25

Kernel	Operation	FLOPs / element	Compulsory bytes / element (f32)	Arithmetic intensity (FLOP/B)
Dot (omp simd)	same, reassociation allowed	2	8	0.25
Elementwise multiply	z[i] = x[i]·y[i]	1	12	0.083
3-point stencil	y[i] = a·x[i−1] + b·x[i] + c·x[i+1]	5	8	0.625

These kernels were chosen because they are simple enough that their arithmetic and memory traffic can be counted exactly, yet each stresses a different property. SAXPY and element-wise multiply are pure streaming kernels with very low arithmetic intensity, so they expose memory-bandwidth limits. The dot product is a reduction, in which every iteration depends on the running sum, so it tests whether the compiler can break a loop-carried dependency. The 3-point stencil reuses each input element three times and performs five floating-point operations per element, giving it the highest arithmetic intensity of the set.

The dot product appears twice. The plain version must keep the strict left-to-right order of floating-point additions, because under IEEE 754 rounding a different order can give a different result. The `omp simd` version uses `#pragma omp simd reduction(+:s)` to tell the compiler that reordering is acceptable, which lets it keep several partial sums in separate vector lanes. Comparing the two isolates the cost of the ordering constraint.

1.3 Goals

The project had five goals: to measure the speedup of auto-vectorized code over scalar code under realistic conditions rather than in an idealized microbenchmark; to distinguish compute-bound behavior, where wider vectors help, from memory-bound behavior, where they do not; to quantify how data type, alignment, loop remainders, stride and working-set size change the result; to verify from compiler reports and disassembly which loops were actually vectorized; and to interpret the results using arithmetic intensity, measured bandwidth and a roofline model.

2. Pre-experiment hypothesis

#	Prediction	Reasoning
H1	Speedup ≈ lane count in L1 (SSE2 ~4×, AVX2 ~8×); 2–4× in L2; 1.3–2× in LLC; ~1.0–1.2× in DRAM	Vector units vs per-level bandwidth ceilings
H2	Plain FP dot not sped up; omp-simd dot ≥ lane count; int32 dot vectorizes without a pragma	IEEE addition order; dependency chain; integer associativity
H3	Stencil keeps more speedup than SAXPY beyond L1	Higher arithmetic intensity, neighbour reuse
H4	f64 ≈ half the f32 L1 speedup; same GiB/s in DRAM; int32 multiply costs more	Half the lanes; bytes per element; vpmulld cost

#	Prediction	Reasoning
H5	Misalignment penalty only when vectors cross cache lines, fading beyond L1; remainder loop adds a tail cost	Line-split loads/stores; scalar or vector epilogue
H6	Useful bandwidth ∝ 1/stride; little or no SIMD benefit for strided/gather access	One cache line per element; shuffle/gather cost
H7	All kernels sit under the bandwidth roof	Arithmetic intensity ≤ 0.63 FLOP/B, far below ridge points

The predictions rest on a few principles. H1 follows from lane count in L1, where the vector units are the limit, and from each lower cache level supplying less bandwidth, so that a kernel moving 12 bytes per element runs out of data before it runs out of arithmetic. H2 follows from IEEE addition order: the plain dot product must add each product to the sum in sequence, so its speed is set by the latency of one floating-point addition whatever the vector width; permitting reassociation removes that chain, and integer addition is associative, so the compiler may reorder it freely. H3 follows from the stencil's higher arithmetic intensity and its reuse of neighbouring elements, which make it less sensitive to bandwidth. H4 follows from float64 having half as many lanes per register and twice as many bytes per element, and from the higher cost of packed 32-bit integer multiplication (`vpmulld`). H5 follows from how loads and stores that span two cache lines are executed, and from the extra code needed for the final few elements. H6 follows from strided elements using only part of each 64-byte cache line, so useful bandwidth falls in proportion to stride. H7 follows from every kernel's arithmetic intensity (at most 0.63 FLOP/byte) lying far below any plausible ridge point.

The predictions were written for a generic x86-64 machine and were not changed for the Ryzen 5 3600 before the run. Zen 2 does not support AVX-512, so the AVX-512 predictions do not apply and are omitted here.

3. Experimental setup and methodology

3.1 Hardware and software

Item	Value
CPU	AMD Ryzen 5 3600 (Zen 2), 6 cores / 12 threads
ISA	SSE2–SSE4.2, AVX, AVX2, FMA (no AVX-512)
Caches	L1d 32 KiB, L2 512 KiB per core; L3 16 MiB per CCX (32 MiB total); measured LLC knee ≈ 13 MiB
Memory	32 GB
OS	Windows 11 Pro, build 26200 (native, no VM)

Author: Brendan Bianchi

Item	Value
Toolchain	MSYS2 UCRT64, g++ 16.2.0 (Rev4)
Python	MSYS2 Python 3.14.7 with numpy 2.5.3, pandas 3.0.5, matplotlib 3.11.2
Timer	QueryPerformanceCounter, 100 ns resolution
Measured clock	3.97–4.05 GHz across all runs (median 4.01 GHz)
Frequency policy	Windows power plan Balanced
SMT	On (2 threads per core)
Pinning	Logical CPU 10 (first thread of core 5), HIGH priority class

3.2 Build variants

Variant	Kernel flags (g++)	Purpose
scalar	-O3 -fno-tree-vectorize -fno-tree-slp-vectorize	Baseline, no SIMD
scalar_native	-O3 -march=native, vectorizers off	Isolates newer ISA (FMA) from vector width
sse2	-O3 -fopenmp-simd	Default 128-bit auto-vectorization
avx2	-O3 -march=x86-64-v3 -fopenmp-simd	256-bit AVX2 + FMA, generic tuning
native	-O3 -march=native -fopenmp-simd	Tuned for Zen 2 (-march=znver2)

All kernels: `-std=c++17 -falign-functions=64 -falign-loops=64`. Windows extras: `-Wa,-muse-unaligned-vector-move`, `-static`.

The benchmark harness (timing, data initialization and correctness checks) is compiled with the same fixed flags into every binary; only the kernel source file is compiled with the variant's flags. Any difference between builds therefore comes from the kernels themselves, not from, for example, a faster timing loop or differently vectorized initialization code.

The kernels are declared `noinline` and placed in a separate source file. Otherwise, the compiler could inline a kernel into the timing loop and then hoist or eliminate work that is identical across repetitions, which would make the measured time meaningless.

All kernels are compiled with `-falign-functions=64 -falign-loops=64`. During development, the speed of an unchanged scalar loop varied by up to 1.6× depending only on where the linker placed it relative to 64-byte boundaries (Section 13, A2). Aligning every function and loop to 64 bytes fixes their placement, so this front-end effect cannot be mistaken for a vectorization effect.

On 64-bit Windows, GCC does not guarantee the 32-byte stack alignment that aligned AVX instructions require (GCC bug 54412), which can crash a program when AVX registers are spilled to the stack. The assembler option `-Wa,-muse-unaligned-vector-move` replaces aligned vector moves with unaligned

ones, which are equally fast on aligned data and safe on misaligned data. Binaries are linked with `-static` so they run outside the MSYS2 shell.

3.3 Timing and statistics

Parameter	Value
Warm-up	≥ 2 calls and ≥ 20 ms
Minimum sample length	10 ms (2 ms in tail study)
Rounds	3 independent rounds, job order shuffled each round
Samples per round	7 (3 for DRAM sizes, 5 in tail study)
Reported statistic	Median of per-round medians; minimum for alignment and tail
Error bars	Inter-quartile range of round medians
Speedup interval	[scalar p25 / variant p75, scalar p75 / variant p25]

Timing on a desktop operating system suffers occasional bursts of interference from background tasks and interrupts, which make individual samples slower but never faster. The median is insensitive to such outliers. Running each configuration in three independent rounds, with the job order shuffled each time, spreads any longer interference episode across different configurations, and taking the median of the three round medians discards a single bad round. Error bars show the inter-quartile range of the round medians.

Alignment and tail costs are small, deterministic effects of a few nanoseconds per call. For these, the minimum time over all samples is reported, because it is the measurement least disturbed by interference and best reflects the intrinsic cost of the code.

Cycles per element are computed as time per element multiplied by the core clock frequency. The clock is measured directly by timing a chain of dependent integer additions, each of which takes exactly one cycle; it stayed between 3.97 and 4.05 GHz throughout the run.

3.4 Data initialization and working sets

Each array is allocated on a 4 KiB boundary and its start is then shifted by a chosen byte offset, so alignment can be controlled exactly. Sixty-four guard elements follow each array; they are checked after every correctness test to detect out-of-bounds writes.

Arrays are filled with deterministic pseudo-random values from an xorshift generator: floating-point values uniformly in [−1, 1] and integers in [−8, 8]. A fixed seed makes every run reproducible, and the bounded range avoids overflow and denormal numbers. The arrays are written before timing begins, so the operating system has already mapped every page (first touch) and page faults are not timed.

The working-set size was swept from 4 KiB to 1 GiB with two points per octave. This range places the data in turn in L1 (32 KiB), L2 (512 KiB), the L3 cache (16 MiB per core complex) and main memory.

SAXPY updates `y` in place, so repeating it with a fixed `a` would make the values grow without bound until they overflow, which would change the timing. Alternating the sign of `a` on each call keeps the values bounded while performing the same work.

4. Correctness validation

Item	Value
Checks run	9,750 (5 builds × all kernels, data types and access patterns × 26 sizes × 3 offsets)
Failures	0 (all 9,750 checks PASS in `verification/correctness.csv`)
Sizes	0, 1, 2, 3, 5, 7, 8, 9, 15, 16, 17, 31, 32, 33, 63, 64, 65, 100, 127, 128, 129, 1000, 1023, 1024, 1025, 4099
Base offsets	0, 4, 12 bytes
Reference	long double (FP) or exact modular arithmetic (int32), vectorization off

The test sizes include zero and other very small sizes, sizes one below, at and one above multiples of the vector widths (8, 16, 32, 64, 128, 1024), and odd sizes such as 4099. Together they exercise every code path the compiler may generate: the main vector loop, the vectorized epilogue, the scalar remainder and the empty loop. Base offsets of 0, 4 and 12 bytes test aligned and misaligned starting addresses.

Each result is compared with a reference computed with vectorization disabled; floating-point references accumulate in `long double`. Element-wise kernels must match within a per-element bound of a few rounding units. Reductions must lie within the worst-case error bound for summing n terms, which grows with n because the vectorized and scalar code add in different orders. Integer results must match exactly, using wrap-around arithmetic.

After each call, the guard elements beyond each array and the untouched gaps between arrays are checked for modification, so a kernel that wrote past the end of its output would fail even if its results were correct.

All 9,750 checks passed. One limitation is that the reduction bound is a worst-case bound and therefore loose: a small error in a single term of a long floating-point sum could stay within it and go undetected.

5. Vectorization verification

Two independent sources were used to decide whether each loop was vectorized. The first is g++'s optimization record, produced with `-fsave-optimization-record`, which lists every loop the vectorizer transformed and the vector width it chose. The second is the disassembly of each kernel, produced with `objdump` and inspected for packed instructions (such as `vmulps` or `vfmadd231ps`) operating on 128-bit `xmm` or 256-bit `ymm` registers.

The disassembly is treated as ground truth because it shows the instructions that actually execute. The compiler's report says only that a transformation was applied. As Section 12.1 shows, g++ reports the plain dot product as vectorized even though the generated loop still adds the products one at a time.

Author: Brendan Bianchi

Function	scalar	sse2	avx2	native
saxpy_f32	scalar	128-bit	256-bit	256-bit
dot_f32	scalar	128-bit multiply, serial scalar adds	256-bit multiply, serial scalar adds	256-bit multiply, serial scalar adds
dotsimd_f32	scalar	128-bit	256-bit	256-bit
dot_i32	scalar	128-bit	256-bit	256-bit
stencil_f32	scalar[1]	128-bit	256-bit	256-bit
saxpy_s2_f32, saxpy_s4_f32	scalar	128-bit + shuffles	256-bit + shuffles	256-bit + shuffles
saxpy_s8_f32	scalar	scalar (not vectorized)	scalar (permutation not supported)	256-bit + shuffles
saxpy_s16_f32	scalar	scalar (not vectorized)	scalar (not vectorized)	scalar (not vectorized)
saxpy_gather_f32	scalar	128-bit	128-bit	128-bit

Vectorization summary. Disassembly verdict for the hottest loop in each build (from `verification/vectorization_summary.md`). Every function vectorized by the disassembly is also reported as vectorized by g++. [1] The automatic classifier labels this cell vector-128 because g++ uses `movups` to copy registers; the loop contains no packed arithmetic, so it is scalar.

Excerpt 1. SAXPY inner loop, `avx2` build (from `verification/hot_loops.md`).

```
40: vmovups         (%r8,%rax,1),%ymm0
46: vfmadd213ps     (%r9,%rax,1),%ymm2,%ymm0
4c: vmovups         %ymm0,(%r9,%rax,1)
52: add             $0x20,%rax
56: cmp             %rcx,%rax
59: jne             40 <saxpy_f32+0x40>
```

Excerpt 2. Plain dot product inner loop, `avx2` build.

```
140: vmovups         (%r8,%rax,1),%ymm1
146: vmulps          (%rdx,%rax,1),%ymm1,%ymm1
14b: add             $0x20,%rax
14f: vaddss          %xmm1,%xmm0,%xmm0
153: vshufps         $0x55,%xmm1,%xmm1,%xmm3
158: vshufps         $0xff,%xmm1,%xmm1,%xmm2
15d: vaddss          %xmm3,%xmm0,%xmm0
161: vunpckhps       %xmm1,%xmm1,%xmm3
165: vextractf128    $0x1,%ymm1,%xmm1
16b: vaddss          %xmm3,%xmm0,%xmm0
16f: vaddss          %xmm2,%xmm0,%xmm0
173: vshufps         $0x55,%xmm1,%xmm1,%xmm2
178: vaddss          %xmm1,%xmm0,%xmm0
17c: vaddss          %xmm2,%xmm0,%xmm0
180: vunpckhps       %xmm1,%xmm1,%xmm2
```

Author: Brendan Bianchi

```
184: vshufps       $0xff,%xmm1,%xmm1,%xmm1
189: vaddss        %xmm2,%xmm0,%xmm0
18d: vaddss        %xmm1,%xmm0,%xmm0
191: cmp           %rax,%r10
194: jne           140 <dot_f32+0x40>
```

Excerpt 1 shows the AVX2 SAXPY loop. It has six instructions: a 256-bit load of eight `x` values into `ymm0`, one `vfmadd213ps` that multiplies them by `a` and adds eight `y` values read directly from memory, a 256-bit store back to `y`, and the pointer increment, compare and branch. Each pass therefore processes eight elements, whereas the scalar build's loop processes one element per pass with separate `mulss` and `addss` instructions.

Excerpt 2 shows the plain dot product in the same build. The eight products are computed with one packed `vmulps`, but g++ then takes them apart with `vshufps`, `vunpckhps` and `vextractf128` and adds them to the sum one at a time with eight scalar `vaddss` instructions. Each `vaddss` depends on the previous one, so the loop still performs one dependent addition per element, exactly as the scalar code does. This is how the compiler can report the loop as vectorized while its speed does not change.

6. Results: scalar vs auto-vectorized builds across working-set sizes

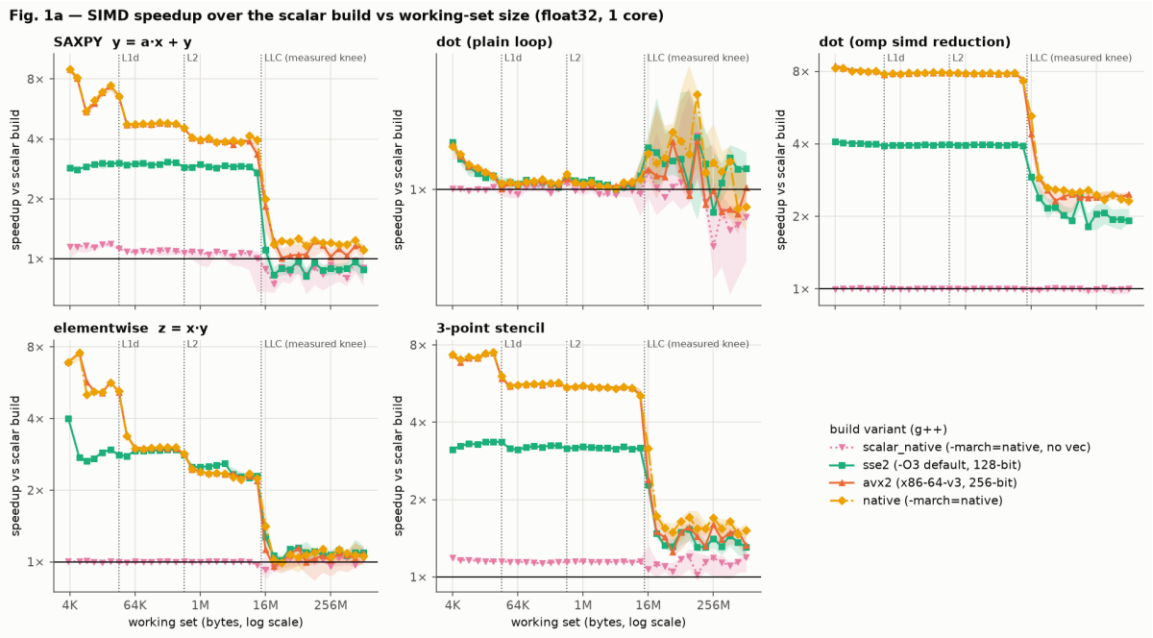


Figure 1a. Speedup over the scalar build vs working-set size, float32.

Author: Brendan Bianchi



Figure 1b. Achieved GFLOP/s vs working-set size.

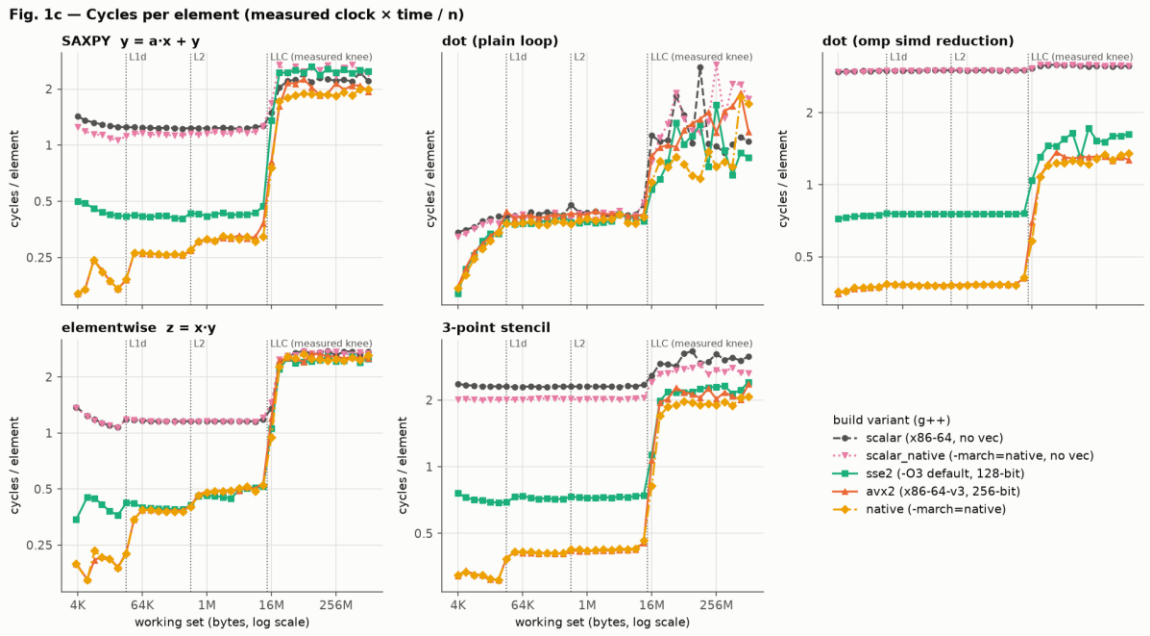


Figure 1c. Cycles per element vs working-set size.

Table 1. Speedup vs scalar build, float32, median [IQR interval]. Scalar GFLOP/s in L1 shown for reference.

Kernel	Scalar GFLOP/s (L1)	Build	L1 (16 KiB)	L2 (256 KiB)	LLC (4 MiB)	DRAM (1 GiB)

Author: Brendan Bianchi

Kernel	Scalar GFLOP/s (L1)	Build	L1 (16 KiB)	L2 (256 KiB)	LLC (4 MiB)	DRAM (1 GiB)
SAXPY	6.26	sse2	3.02×	3.06×	2.90×	0.88× [0.78–0.99]
SAXPY		avx2	6.84×	4.76×	3.75×	1.13× [0.95–1.19]
SAXPY		native	6.89×	4.79×	3.94×	1.11×
Dot (plain)	2.67	avx2	1.02×	1.01×	1.00×	1.00×
Dot (omp simd)	2.69	sse2	3.99×	3.96×	3.96×	1.92×
Dot (omp simd)		avx2	8.03×	7.88×	7.82×	2.47×
Multiply	3.66	avx2	5.13×	3.00×	2.34×	1.09×
Stencil	8.64	avx2	7.42×	5.68×	5.46×	1.33×
Stencil		native	7.44×	5.68×	5.42×	1.52×

Figure 1a shows the same pattern for every vectorizable kernel: the speedup is highest when the working set fits in L1 and falls at each cache boundary as data must come from a slower level.

In L1, SSE2 achieves about 3.0× for SAXPY and AVX2 about 6.8×, against ideal values of 4× and 8×. The shortfall comes from work that does not scale with vector width. The AVX2 SAXPY loop takes about 1.5 cycles per eight-element iteration rather than one, because each iteration issues two loads, one store, one FMA and the loop-control instructions, and g++ does not unroll the loop to overlap this overhead. The scalar loop, meanwhile, already completes about 0.8 elements per cycle.

The `native` build, tuned for Zen 2, performs within a few percent of the generic `avx2` build for all contiguous kernels, because both generate essentially the same AVX2 code for these simple loops.

The plain dot product runs at about 1.0× in every build and at every size. Figure 1c shows it at about 3 cycles per element in all builds, which equals the latency of one floating-point addition on Zen 2: each addition must wait for the previous one to finish. The `omp simd` version keeps eight independent partial sums in the lanes of one vector register, so the same 3-cycle chain advances eight elements at a time, giving 8.0×, exactly the lane count.

The `scalar_native` build, which uses `-march=native` with vectorization disabled, runs SAXPY and the stencil 1.15–1.18× faster than the baseline scalar build in L1. This gain comes only from newer instructions such as FMA, which combines a multiply and an add, and shows how much of the vector builds' speedup is not due to vector width.

In DRAM, the SAXPY speedups are about 1× and their intervals are wide (0.78–0.99 for SSE2). The SSE2 value of 0.88× should therefore be read as no measurable difference, not as SSE2 being slower than scalar.

Author: Brendan Bianchi

7. Results: locality sweep (SAXPY, float32)

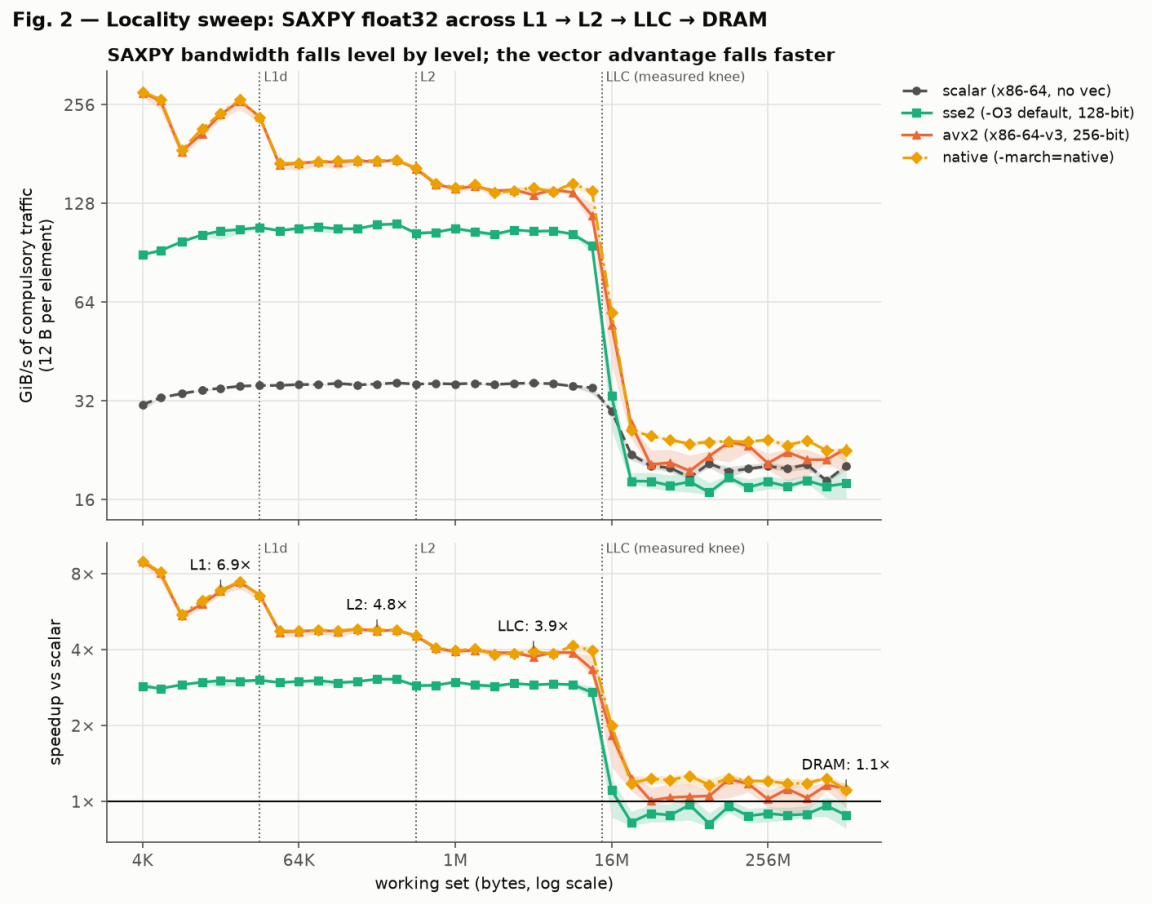


Figure 2. SAXPY bandwidth (GiB/s) and speedup vs working-set size, with cache boundaries marked.

Table 2. Measured single-core bandwidth (GiB/s). The roof is the best sustained value of any streaming kernel in any vector build.

Stream	L1	L2	LLC	DRAM
Read-only (native)	40.5	39.7	39.7	20.2
Write-only (native)	113.2	117.0	81.9	10.1
Roof (SAXPY, native)	253.0	172.6	141.5	24.0

Table 3. AVX2 SAXPY speedup by level: 6.84× (L1) → 4.76× (L2) → 3.75× (LLC) → 1.13× (DRAM).

Figure 2 shows the AVX2 SAXPY speedup falling in steps. In L1 (6.8×) the limit is the core itself: how fast it can issue the loop's loads, stores and arithmetic. In L2 (4.8×) it is the rate at which L2 can refill L1, about 173 GiB/s. In the L3 cache (3.8×) it is L3 bandwidth, about 142 GiB/s. In DRAM (1.1×) it is main-memory bandwidth, only 24 GiB/s for a single core.

The largest drop on this processor is from the L3 cache to DRAM, not from L1 to L2. One Zen 2 core can stream about 140 GiB/s from its L3, more than half its L1 rate, but only about 24 GiB/s from DRAM, a

sixfold fall. The vector builds therefore keep most of their advantage until the working set outgrows the L3 cache.

The scalar build is almost flat from L1 to the L3 cache at about 6.3–6.5 GFLOP/s. It needs only about 35 GiB/s, which every cache level can supply, so its speed is set by the number of instructions it executes, not by where the data is.

The measured knee where data spills from the L3 cache is about 13 MiB, smaller than the 16 MiB of L3 in each core complex. Zen 2's L3 is a victim cache shared by the three cores of a complex, and the benchmark's other arrays, its code and the operating system also occupy part of it, so less than the full capacity is available to the working set.

The read-only probe reaches only about 40 GiB/s in L1, far below SAXPY's 253 GiB/s. The probe sums the values it reads, and that sum forms a dependency chain limited by addition latency, so it measures the reduction speed rather than L1 bandwidth. This is why the roofline uses the best streaming kernel, SAXPY, for its bandwidth roofs.

8. Results: alignment and tail handling

8.1 Alignment

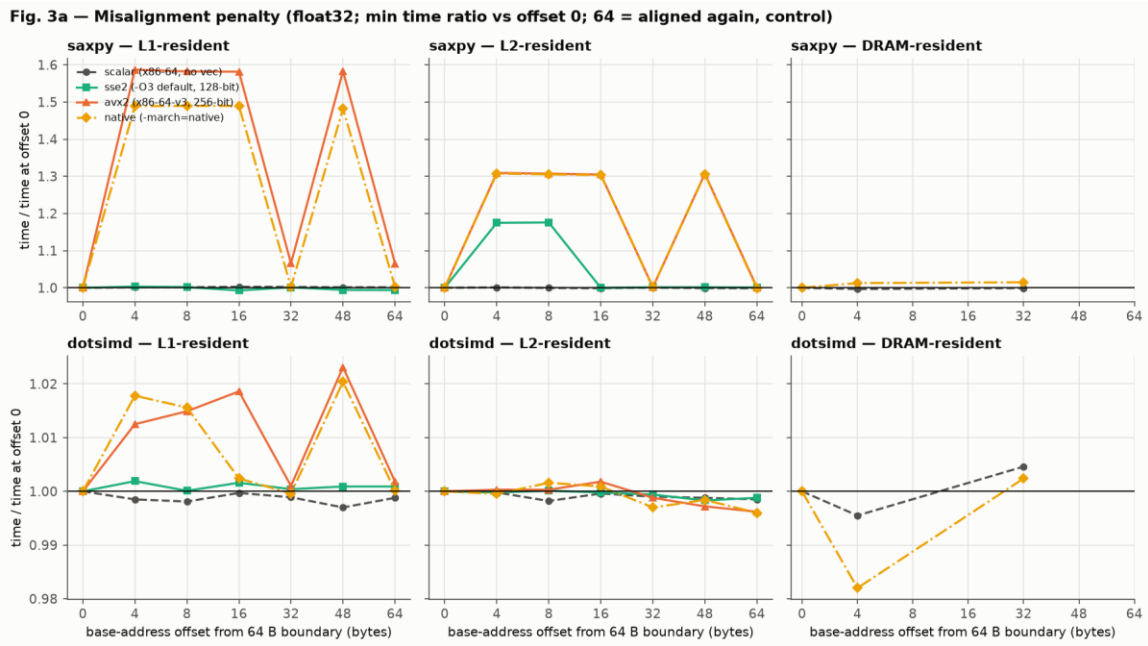


Figure 3a. Time ratio vs base-address offset (1.0 = same as aligned).

Table 4. Slowdown vs offset 0 (minimum-time ratio; >1 = slower). Offset 64 is aligned again (control).

Kernel / level / build	4 B	8 B	16 B	32 B	48 B	64 B
SAXPY L1 avx2	1.59	1.58	1.58	1.07	1.58	1.07
SAXPY L1 native	1.49	1.49	1.49	1.00	1.48	1.00

Author: Brendan Bianchi

Kernel / level / build	4 B	8 B	16 B	32 B	48 B	64 B
SAXPY L2 avx2	1.31	1.31	1.30	1.00	1.31	1.00
SAXPY L2 sse2	1.17	1.18	1.00	1.00	1.00	1.00
SAXPY L1/L2 scalar	1.00	1.00	1.00	1.00	1.00	1.00
dotsimd L1/L2 (all builds)	1.00–1.02	1.00–1.02	1.00–1.02	1.00	1.00–1.02	1.00
SAXPY DRAM native	1.01	—	—	1.01	—	—

The data show that misalignment costs time only when a vector access crosses the boundary between two 64-byte cache lines. With 32-byte AVX2 vectors this happens at any offset that is not a multiple of 32 bytes; at offsets of 32 and 64 bytes each vector lies within one line and the penalty disappears.

For AVX2 SAXPY, misaligned offsets cost about 1.5–1.6× in L1 and 1.3× in L2, and nothing at 32 bytes. With 16-byte SSE2 vectors only offsets of 4 and 8 bytes cause splits, and the resulting penalty (1.17×) appears in L2 but not in L1. The scalar build never pays a penalty, because a 4-byte element never straddles a cache line.

The load-only `dotsimd` kernel shows at most a 2% penalty, while SAXPY, which also stores, pays up to 59%. This suggests that Zen 2 handles split loads almost for free and that stores spanning two cache lines are the expensive operation.

In DRAM there is no measurable penalty, because the same cache lines must be transferred from memory whatever the offset, and that transfer dominates the time.

g++ did not add a peeling loop to align the main loop, so the misaligned accesses were actually executed and the measurements reflect the hardware cost.

The aligned control offsets for AVX2 SAXPY in L1 show 1.07 rather than 1.00. An aligned access cannot cause a split, so this 7% is the noise floor of the measurement and sets the threshold for treating a difference as real.

8.2 Tail (remainder) handling

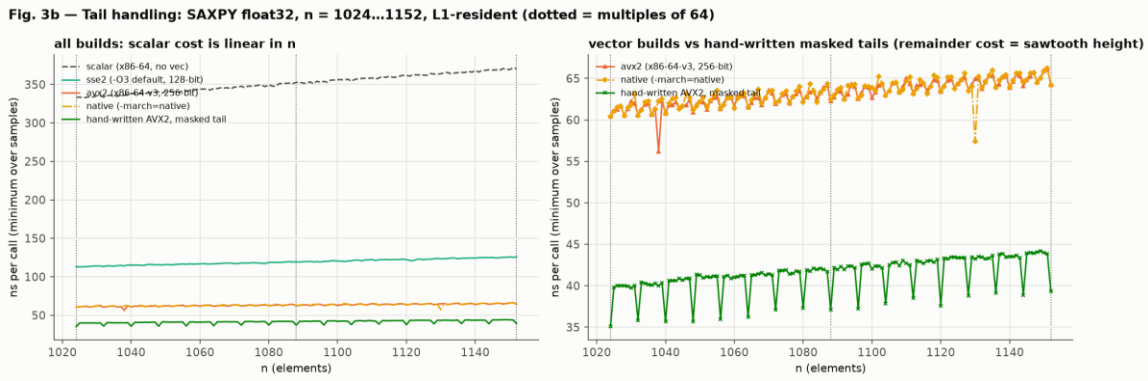


Figure 3b. SAXPY time per call for n = 1024–1152, L1-resident.

Table 5. SAXPY, L1-resident, ns per call (minimum over samples).

Author: Brendan Bianchi

Build	n = 1024	Worst over 1025–1087	n = 1088	Worst extra vs n = 1024
scalar	333.3	351.8	352.7	18.5
sse2	113.0	119.7	119.3	6.6
avx2	60.4	64.2	62.3	3.7
native	60.4	64.4	62.5	3.9
Hand-written AVX2 with masked tail	35.1	42.1	37.1	7.0

Figure 3b shows only a small ripple as n increases from 1024 to 1152, without the large staircase a long scalar remainder loop would produce. g++ handles the remainder with a vectorized epilogue that processes leftover elements with shorter vectors, and because the main loop is not unrolled, at most seven elements are ever left over in the AVX2 build.

The worst extra cost of a remainder in the AVX2 build is 3.7 ns, about 6% of the 60 ns call at n = 1024. The remainder does add a cost, as H5 predicted, but the cost is small and matters little for arrays longer than a few hundred elements.

The hand-written AVX2 kernel with a masked tail is about 1.7× faster overall (35.1 ns versus 60.4 ns), even though its tail costs slightly more. Its main loop is unrolled four times, which spreads the loop overhead over 32 elements and keeps more loads and stores in flight. For these kernels, g++'s main-loop code, not its remainder handling, is the bigger opportunity for improvement.

9. Results: stride and gather

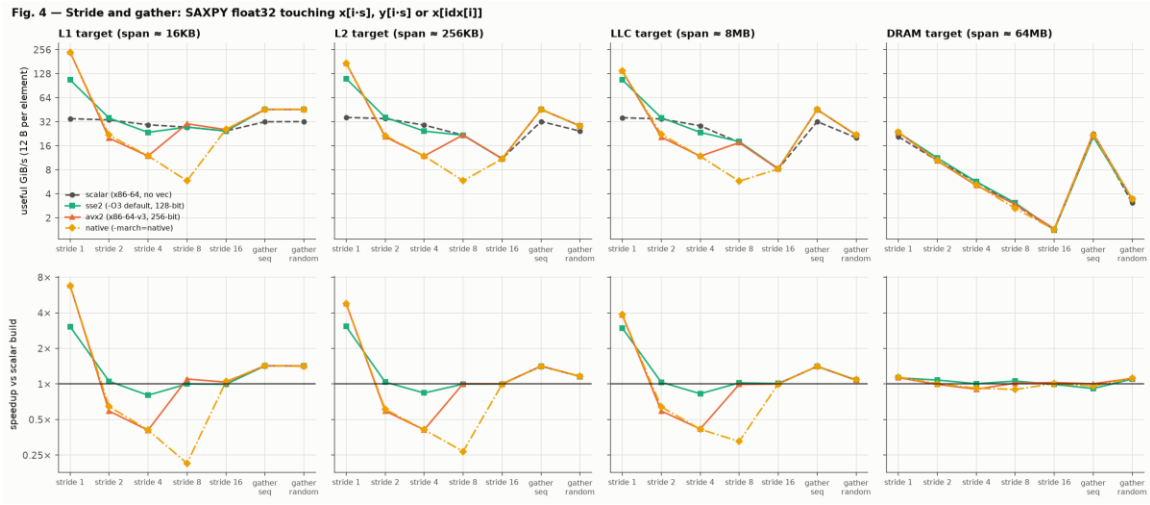


Figure 4. Useful bandwidth and speedup by access pattern, per memory level.

Table 6. Useful bandwidth in GiB/s (speedup vs scalar), L1 and DRAM.

Author: Brendan Bianchi

Pattern	L1 scalar	L1 avx2	L1 native	DRAM scalar	DRAM avx2	DRAM native
Stride 1	35.1	240.8 (6.86×)	237.4 (6.76×)	20.9	23.6 (1.13×)	23.9 (1.15×)
Stride 2	33.8	20.0 (0.59×)	21.8 (0.65×)	10.5	10.4 (0.99×)	10.6 (1.01×)
Stride 4	29.4	12.0 (0.41×)	12.0 (0.41×)	5.7	5.1 (0.90×)	5.2 (0.92×)
Stride 8	27.5	30.3 (1.10×)	5.9 (0.21×)	2.9	3.0 (1.02×)	2.6 (0.90×)
Stride 16	24.6	25.5 (1.04×)	25.8 (1.05×)	1.4	1.5 (1.03×)	1.4 (1.01×)
Gather, sequential index	32.1	45.9 (1.43×)	45.8 (1.43×)	22.7	22.8 (1.01×)	21.8 (0.96×)
Gather, random index	32.2	45.9 (1.42×)	45.5 (1.41×)	3.1	3.5 (1.12×)	3.5 (1.12×)

Excerpt 3. Stride-8 SAXPY inner loop: `avx2` build (left scalar by g++) vs `native` build (vectorized). Disassembled from `build/<variant>/kernels.o`.

```
; avx2 build: one element per iteration
1a80: vmovss         (%r8,%rax,1),%xmm0
1a86: vfmadd213ss    (%r9,%rax,1),%xmm1,%xmm0
1a8c: vmovss         %xmm0,(%r9,%rax,1)
1a92: add            $0x20,%rax
1a96: cmp            %rax,%rcx
1a99: jne            1a80

; native build: eight elements per iteration, 58 instructions:
;   28 x vpermd, 14 x vblendps   (assemble 8 strided x and 8 strided y values)
;    1 x vfmadd231ps             (the useful work)
;    4 x vmovss, 6 x vextractps, 1 x vextractf128  (load/store single elements)
;    add, add, cmp, jne
1ac0: vpermd         (%rdx),%ymm3,%ymm5
1ac5: vpermd         0x20(%rdx),%ymm2,%ymm0
...
1bd6: vfmadd231ps    %ymm4,%ymm5,%ymm0
1bdb: vmovss         %xmm0,-0x100(%rax)
1be3: vextractps     $0x1,%xmm0,-0xe0(%rax)
...
1c24: jne            1ac0
```

In DRAM, useful bandwidth halves with each doubling of the stride: 20.9, 10.5, 5.7, 2.9 and 1.4 GiB/s for strides 1, 2, 4, 8 and 16. Memory delivers whole 64-byte cache lines, so at stride 2 half of each line is wasted, and at stride 16 (64 bytes) each useful 4-byte element costs a full line. Total traffic stays roughly constant while the useful fraction shrinks.

For strides 2 and 4, vectorization makes the loop slower: the `avx2` and `native` builds reach only 0.41–0.65× of scalar in L1. AVX2 has no strided load, so g++ loads several contiguous 256-bit blocks and uses `vpermd` and `vblendps` to collect the needed elements into one register; after a single `vfmadd` it writes

the eight results back one at a time with `vmovss` and `vextractps`. At stride 4 this takes 18 permute and blend instructions and eight single-element stores for every eight elements, which costs more than the scalar loop, which simply loads, computes and stores each element directly.

The stride-8 case shows how fragile this decision is (Excerpt 3). The generic `avx2` build does not vectorize the loop at all: g++'s report says the required permutation is not supported, so it emits a scalar loop with one `vfmadd213ss` per element. That loop runs at 1.10× of scalar for the same reason as the `scalar_native` build: FMA replaces a separate multiply and add. The Zen 2–tuned `native` build does vectorize the loop, but needs 28 `vpermd` and 14 `vblendps` instructions plus ten single-element stores and extracts for every eight elements, and runs at only 0.21× of scalar.

Gather patterns run about 1.4× faster than scalar in cache in every vector build. The loads stay scalar because g++ emulates the gather, but the arithmetic and stores are vectorized. Random gathers in DRAM reach only about 3 GiB/s against about 23 GiB/s for sequential indices, because each element needs a separate memory access whose latency cannot be hidden, and SIMD changes this only slightly (1.12×).

10. Results: data-type comparison

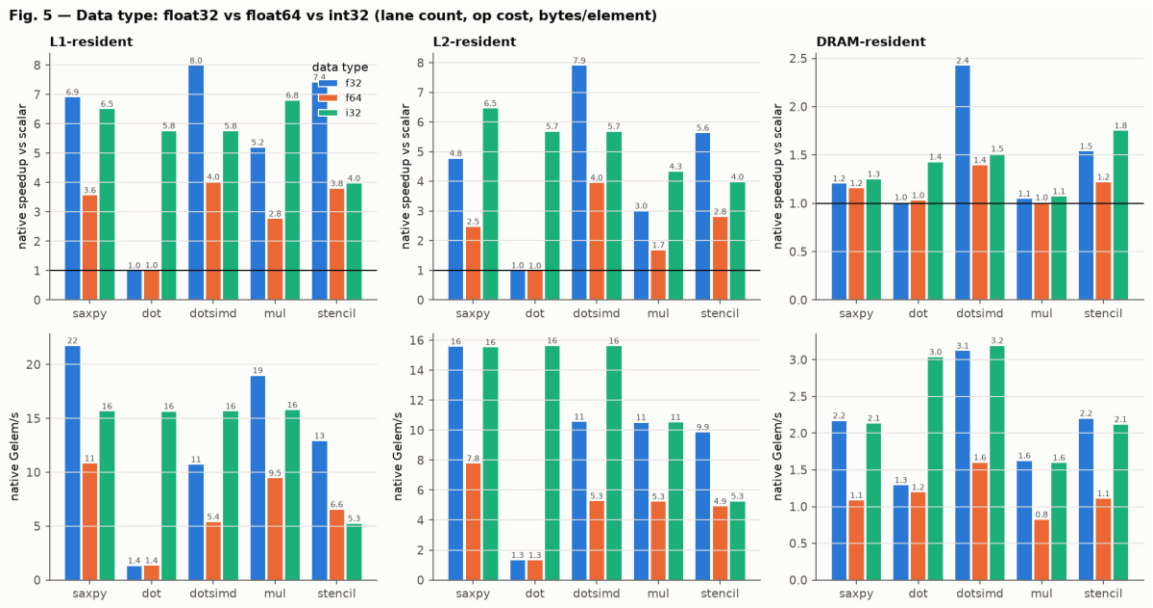


Figure 5. Speedup and element throughput by kernel and data type.

Table 7. `native` build speedup vs scalar (L1 / L2 / DRAM).

Kernel	float32	float64	int32
SAXPY	6.92 / 4.79 / 1.21×	3.58 / 2.48 / 1.16×	6.52 / 6.47 / 1.25×
Dot (plain)	1.01 / 1.01 / 1.01×	1.02 / 1.00 / 1.03×	5.77 / 5.70 / 1.43×
Dot (omp simd)	8.00 / 7.92 / 2.43×	4.03 / 3.96 / 1.40×	5.77 / 5.69 / 1.52×

Author: Brendan Bianchi

Kernel	float32	float64	int32
Multiply	5.21 / 3.01 / 1.05×	2.78 / 1.69 / 1.01×	6.81 / 4.35 / 1.08×
Stencil	7.43 / 5.65 / 1.54×	3.81 / 2.82 / 1.22×	3.99 / 3.99 / 1.75×

A 256-bit register holds eight float32 values but only four float64 values, so float64 should gain about half as much from vectorization in L1. The data confirm this: SAXPY reaches 3.58× in float64 against 6.92× in float32, and the `omp simd` dot product 4.03× against 8.00×.

Each float64 element occupies twice as many bytes, so when DRAM bandwidth is the limit, both types move data at a similar rate in GiB/s and float64 processes about half as many elements per second (SAXPY: 1.09 versus 2.17 Gelem/s).

The plain int32 dot product reaches 5.8× without any pragma, while the floating-point versions stay at 1.0×. Integer addition is associative, so reordering the sum gives an identical result and g++ is free to keep partial sums in separate lanes.

Every int32 kernel with one multiply per element (SAXPY, dot product, multiply) levels off at about 15.7 Gelem/s in L1, about two cycles per eight-element vector. The int32 stencil, with three multiplies per element, reaches 5.3 Gelem/s, about six cycles per vector, and its speedup (4.0×) is well below float32's (7.4×). Both figures point to the packed 32-bit integer multiply (`vpmulld`) as the bottleneck: the measurements imply that Zen 2 completes about one every two cycles, whereas it can issue two floating-point multiplies or FMAs per cycle. The int32 stencil runs at the same speed in L1 and L2, which confirms that computation, not bandwidth, limits it.

Hypothesis H4 is largely confirmed: float64 gets about half the float32 speedup in cache, both types move similar GiB/s in DRAM, and int32 multiplication is the costliest operation.

Author: Brendan Bianchi

11. Roofline interpretation

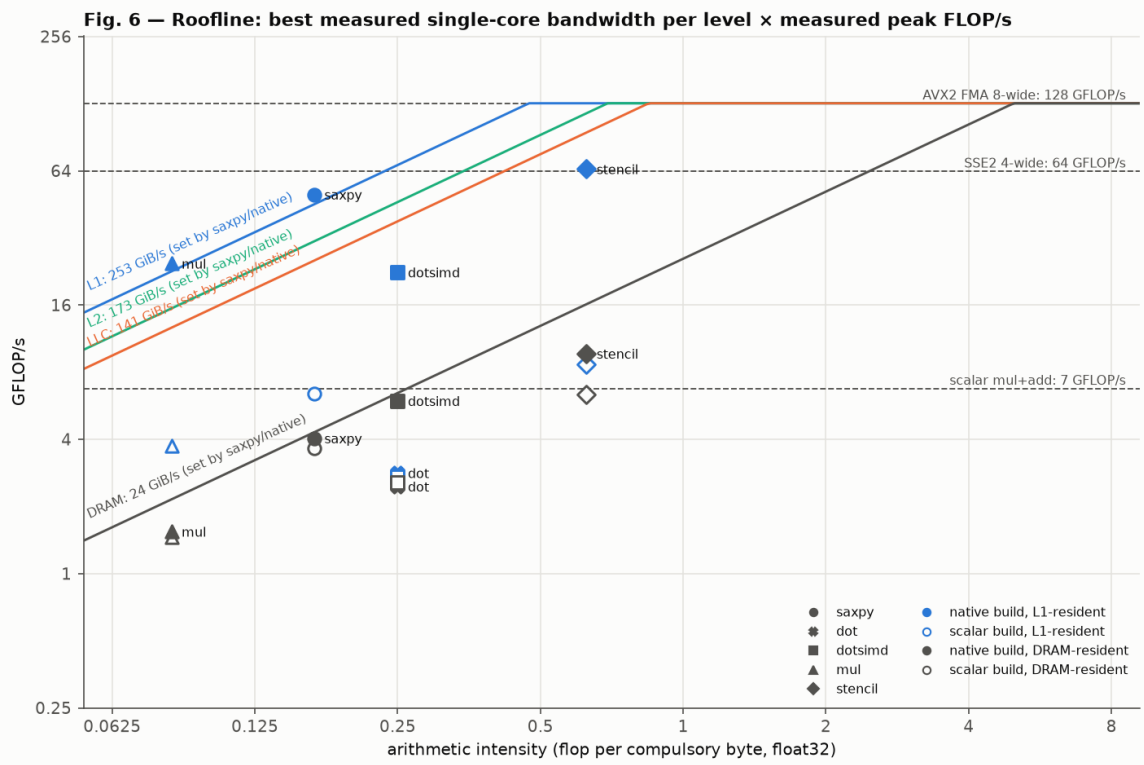


Figure 6. Roofline: measured bandwidth roofs and peak FLOP/s, with kernels plotted at their arithmetic intensity.

Table 8. Measured compute peaks (float32 unless noted).

ISA	GFLOP/s	FLOP / cycle
Scalar mul + add	6.8	1.7
Scalar FMA	16.1	4.1
SSE2 (4-wide mul + add)	63.6	16.0
AVX2 FMA (8-wide)	128.5	32.3
AVX2 FMA, float64	64.3	16.2

Table 9. Ridge points against the AVX2 peak (128.5 GFLOP/s).

Level	Bandwidth roof (GiB/s)	Ridge (FLOP/byte)
L1	253.0	0.47
L2	172.6	0.69
LLC	141.5	0.85

Author: Brendan Bianchi

Level	Bandwidth roof (GiB/s)	Ridge (FLOP/byte)
DRAM	24.0	5.0

The bandwidth roof for each level is the highest sustained bandwidth any streaming kernel reached in any vector build with its working set at that level; SAXPY in the `native` build set every roof. The compute peaks were measured with dedicated loops containing many independent FMA (or multiply and add) chains, so that throughput rather than latency limits them.

The measured AVX2 peak of 32.3 FLOP per cycle matches the theoretical value for Zen 2: two 256-bit FMA units, each processing eight float32 lanes per cycle, with each FMA counting as two floating-point operations (2 × 8 × 2 = 32).

All kernels have an arithmetic intensity between 0.083 and 0.625 FLOP/byte. The DRAM ridge point is 5.0 FLOP/byte, so every kernel is firmly bandwidth-bound when its data comes from memory. Only the stencil, at 0.625 FLOP/byte, reaches the region of the L1 ridge (0.47 FLOP/byte), which is consistent with it keeping the most speedup in cache.

SAXPY would need about 30 times more arithmetic per byte (5.0 ÷ 0.167) before DRAM bandwidth stopped being the limit and wider vectors could help in memory.

The scalar builds sit far below every roof because they are limited by how fast they can issue instructions, not by memory. Vectorization raises this instruction ceiling by up to the lane count, moving each kernel up until it meets the bandwidth roof of the level its data comes from; beyond that point wider vectors cannot help.

12. Negative and failure cases

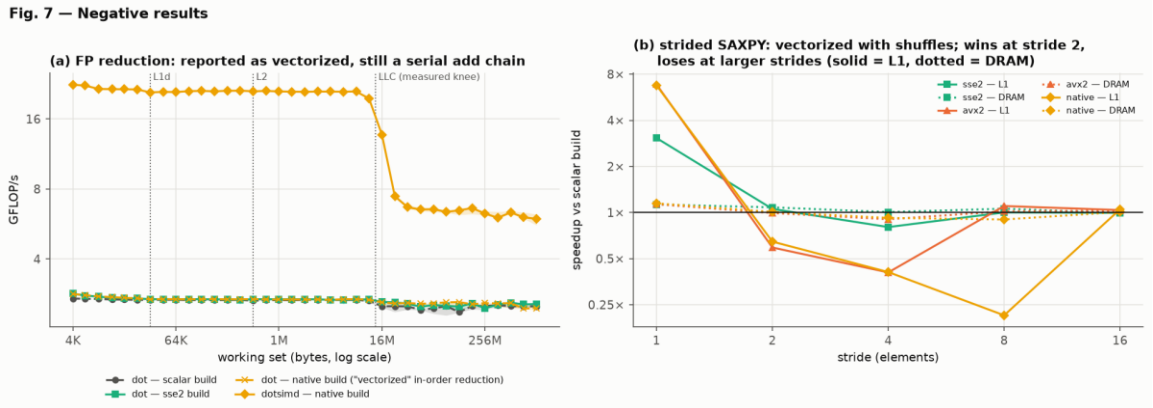


Figure 7. Negative cases: (a) plain vs omp-simd dot product across sizes; (b) strided SAXPY speedup vs stride.

12.1 Reported as vectorized, but no faster: the plain dot product

The plain float32 dot product runs at 1.00–1.02× of scalar in every build and at every working-set size.

g++'s optimization record reports the loop as vectorized. The disassembly shows that the multiplications are done with one packed instruction, but the products are then added to the sum one at a time, in their

original order, with scalar additions. This preserves the exact IEEE result, as required without `-ffast-math`, but leaves the critical path unchanged.

The loop's speed is set by its chain of dependent additions, each taking about 3 cycles on Zen 2. Because the vectorized code still performs one dependent addition per element, its throughput equals the scalar loop's: about 3 cycles per element.

The `omp simd` version may reorder the sum, so it keeps eight independent partial sums, one per lane, and combines them after the loop. The same 3-cycle chain then covers eight elements per step, giving 8.0×. The whole difference between the two versions comes from permission to reorder the additions.

12.2 Vectorized and slower: strided access

For strides 2 and 4, the `avx2` and `native` builds run at 0.41–0.65× of scalar in L1, and the `native` build falls to 0.21× at stride 8. In these cases vectorization made the code slower.

g++'s cost model judged that building vectors from strided elements with shuffles, and writing results back element by element, would still pay off. On Zen 2 the extra shuffle and extract instructions compete for the same execution resources as the useful work, and their cost exceeds the saving from vector arithmetic. The compiler's estimate of these instruction costs does not match this processor.

12.3 Vectorized but memory-bound: DRAM

With eight lanes available, SAXPY gains only 1.11–1.13× and element-wise multiply 1.06–1.09× when their data comes from DRAM.

These kernels perform 0.083–0.167 floating-point operations per byte moved, against a DRAM ridge point of 5.0 FLOP/byte. Even the scalar code comes close to the 24 GiB/s one core can draw from memory, so doing the arithmetic faster only leaves the core waiting for data. Vectorization is working correctly; there is simply nothing left for it to speed up.

13. Anomalies and plausible explanations

#	Observation	Data	Explanation
A1	`native` far slower than `avx2` at stride 8	L1: 0.21× vs 1.10×; L2: 0.27× vs 1.01×	Generic tuning leaves the loop scalar; Zen 2 tuning vectorizes it with 42 shuffles per 8 elements. Evidence: Excerpt 3, `objdump` of `saxpy_s8_f32` in both builds
A2	Code placement changed kernel speed (seen while building the project)	Scalar SAXPY 1.6× slower when its loop crossed a 64-byte boundary on the development machine	Front-end (µop cache / decode) effect; fixed with 64-byte function and loop alignment
A3	Measured LLC knee below	~13 MiB vs 16 MiB	Victim-cache behavior and other

Author: Brendan Bianchi

#	Observation	Data	Explanation
	the per-CCX L3 size		data/code sharing the L3
A4	SAXPY in DRAM shows SSE2 below 1×	0.88× [0.78–0.99]	Few DRAM samples and run-to-run noise; smoke run showed 1.12×
A5	Aligned control offsets not exactly 1.00	AVX2 SAXPY L1 offset 32/64: 1.07	Measurement noise floor
A6	`scalar_native` faster than scalar for SAXPY/stencil without SIMD	1.15–1.18× in L1	Fewer instructions from FMA and memory-operand forms

A1. At stride 8, the Zen 2–tuned `native` build runs at 0.21× of scalar in L1 and 0.27× in L2, while the generic `avx2` build runs at 1.10× and 1.01×. The disassembly (Excerpt 3) explains the difference. Under generic AVX2 tuning, g++ declines to vectorize the loop and emits a scalar FMA loop. Under `-march=znver2`, its cost model accepts a vectorized version built from 42 permute and blend instructions per eight elements. The anomaly is therefore a compiler cost-model decision rather than a hardware effect: the version the compiler expected to be faster is about five times slower than the one it rejected.

A2. During development, on a different machine, an unchanged scalar SAXPY loop ran 1.6× slower in one build only because its position in memory made it cross a 64-byte boundary. This is a front-end effect: the loop no longer fitted the processor's micro-op cache or decoders as efficiently. Aligning every function and loop to 64 bytes removed the effect, and it did not recur in the final runs.

A3. The working set spills from the L3 cache at about 13 MiB rather than at the full 16 MiB of a core complex. Zen 2's L3 is a victim cache shared by three cores, and other data and code also occupy it. A finer sweep around the knee, with the other cores of the complex idle, would test this explanation.

A4. SSE2 SAXPY in DRAM appears at 0.88× of scalar, but its interval is 0.78–0.99 and the smoke run showed 1.12×. DRAM measurements use fewer samples and are more exposed to background memory traffic, so this value is within run-to-run noise.

A5. AVX2 SAXPY in L1 shows a ratio of 1.07 at the aligned 32- and 64-byte offsets, where no cache-line split can occur. This is the noise floor of the alignment measurement.

A6. The `scalar_native` build runs SAXPY and the stencil 1.15–1.18× faster than the baseline scalar build in L1 without using SIMD. The disassembly shows why: with `-march=native`, g++ replaces the scalar build's separate `mulss` and `addss` with a single `vfmadd213ss` that also reads `y` directly from memory, so each element needs fewer instructions.

No frequency reduction was observed under AVX2 load: the core clock measured 4.02 GHz immediately after sustained AVX2 FMA work and 4.03 GHz after scalar work. This contrasts with Intel processors that support AVX-512, which lower their clock during heavy 512-bit work and can lose part of the vector advantage as a result.

Measurement quality was good. In only 0.5% of configurations was one round more than 1.5× slower than another, and the median coefficient of variation across all 2,134 configurations was 4.5%.

Author: Brendan Bianchi

14. Threats to validity

All measurements come from one processor (an AMD Ryzen 5 3600, Zen 2), one compiler (g++ 16.2) and one operating system (Windows 11). Other microarchitectures, such as Intel cores or newer AMD cores with AVX-512, and other compilers, such as Clang or MSVC, may generate different code and give different speedups.

The clock frequency was not fixed and boost was active, although the measured clock stayed between 3.97 and 4.05 GHz. Hardware performance counters were not available on Windows, so cache misses and instruction counts were not measured directly; conclusions about vectorization rest on compiler reports, disassembly and timing.

Bandwidth figures count only compulsory bytes: each input read once and each output written once. A store to a line that is not in cache also requires that line to be read first (write-allocate), so the true traffic for SAXPY and multiply is somewhat higher than reported.

Simultaneous multithreading was enabled, so the sibling logical CPU on the measurement core could run operating-system work during a sample and compete for the core's resources. Medians and repeated rounds limit this effect but do not remove it.

DRAM-sized working sets were measured with fewer samples because each takes much longer, and their intervals are correspondingly wider.

15. Hypothesis vs outcome

#	Outcome	Key evidence
H1	Partly confirmed: L2 and LLC speedups exceeded the predicted ranges	SSE2 3.0×, AVX2 6.8× in L1; AVX2 4.8× in L2 and 3.75× in LLC; ~1.1× in DRAM
H2	Confirmed	Plain dot 1.0×; omp-simd dot 8.0×; int32 dot 5.8× without a pragma
H3	Confirmed	Stencil vs SAXPY (AVX2): L2 5.68 vs 4.76×; LLC 5.46 vs 3.75×; DRAM 1.33 vs 1.13×
H4	Confirmed	f64 SAXPY 3.58× vs f32 6.92× in L1; similar GiB/s in DRAM; int32 stencil 4.0×
H5	Partly confirmed: penalty persisted in L2; tail cost smaller than expected	AVX2 1.5× (L1) / 1.3× (L2) misaligned, none at 32 B or in DRAM; tail ≤ 4 ns
H6	Confirmed	DRAM stride bandwidth halves per doubling; strided 0.21–0.65× in L1
H7	Confirmed	All kernels below the DRAM ridge (5.0 FLOP/B)

16. Conclusion

SIMD vectorization delivers close to its theoretical speedup when three conditions hold: the data is in cache, the access is contiguous, and the computation either has no loop-carried dependency or is allowed to reorder it. Under those conditions, AVX2 sped up SAXPY by 6.8× and a reordered dot product by 8.0× on one Zen 2 core, and on this processor most of that advantage survives through the L3 cache.

The advantage disappears once the working set exceeds the last-level cache, because a single core's DRAM bandwidth holds every low-intensity kernel to about 1.1× whatever the vector width. It also disappears, or reverses, for strided access, where assembling vectors costs more than it saves, and for floating-point reductions that must keep their order.

The most important practical lesson is that a compiler report saying a loop was vectorized does not show that it is faster. The plain dot product was reported as vectorized but ran at scalar speed. Vectorization must be confirmed both by the disassembly and by a measurement against a scalar baseline.

A natural next step would be to repeat the study with a second compiler such as Clang, to collect hardware performance counters (for example with `perf` on Linux) to confirm the bandwidth explanations directly, and to run the kernels on several cores to see whether combined DRAM bandwidth lets vectorization pay off in memory.

17. Reproducibility

Item	Location
Source code	`src/` (kernels, harness, intrinsics, platform layer)
Build / run / plot scripts	`scripts/` (`run_all.sh` runs everything)
Raw data	`results/my-pc/raw/*.csv`
Processed data	`results/my-pc/processed/*.csv`, `summary.md`
Figures	`results/my-pc/figures/*.png`
Environment record	`results/my-pc/sysinfo.json`
Verification	`results/my-pc/verification/`

Command: `bash scripts/run_all.sh --tag my-pc` (MSYS2 UCRT64 shell; full steps in `docs/WINDOWS.md`).

18. References

1. S. Williams, A. Waterman and D. Patterson, Roofline: An Insightful Visual Performance Model for Multicore Architectures, *Communications of the ACM*, 52(4), pp. 65–76, 2009.
2. Free Software Foundation, *Using the GNU Compiler Collection (GCC)*: Optimize Options and Developer Options (`-fopt-info`, `-fsave-optimization-record`). https://gcc.gnu.org/onlinedocs/

3. GCC project, Auto-vectorization in GCC. https://gcc.gnu.org/projects/tree-ssa/vectorization.html
4. GCC Bugzilla, Bug 54412: 32-byte stack alignment for AVX on 64-bit Windows. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=54412
5. A. Fog, *Instruction tables* and *The microarchitecture of Intel, AMD and VIA CPUs*. https://www.agner.org/optimize/
6. AMD, *Software Optimization Guide for AMD Family 17h Models 30h and Greater Processors*, publication 56305.
7. OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, the `simd` construct. https://www.openmp.org/specifications/
8. IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE 754-2019.
9. MSYS2 project. https://www.msys2.org/

Appendix

- A. Full results summary: `results/my-pc/summary.md`.
- B. Vectorization summary: `results/my-pc/verification/vectorization_summary.md`.
- C. Hot-loop disassembly: `results/my-pc/verification/hot_loops.md`.
- D. Raw timing samples and command log: `results/my-pc/raw/`.